Machine Learning for Signal Processing

[5LSL0]

Rik Vullings
Ruud van Sloun
Hans van Gorp
Louis van Harten
Tristan Stevens
Oisín Nolan
Simon Penninga

Assignment 1: Familiarization with Pytorch and Practical Fundamentals

Academic Year 2025-2026

Use of generative AI

You are allowed to use generative AI, specifically to create content (like images, (source) code, video, text or any other kind) and writing. However, you remain solely responsible for the answers you provide and you remain solely responsible for your own learning process. In the written exam, use of generative AI is not permitted.

Grading

The final exercise in each assignment is graded; in total these exercises contribute 20% of your final grade. The other exercises are not graded, but the code and/or understanding you build along the way is typically required to solve the final exercise.

Assignments that require you to code in python are denoted with a icon. You may code in any IDE you like. For your own benefit, keep the code clean, legible, and include lots of comments; code that you create for this assignment can be repurposed for later assignments.

Submission format

Code needs to be collated into three standalone files: `exercise_1.py`, `exercise_2.py` and `exercise_3.py`. Which should create all the figures and tabels. We suggest you include a command line argument `--load` in your scripts, which loads your saved results (`.tar`-files) to generate the figures without retraining everything. Make sure the script produces all to-be-loaded files when called without this argument.

Answers that are not code should be provided as a `.tex`-document. Make sure that the answer to each question is clearly labeled using a `\subsection{1.1a}` command. We will parse everything that is placed between e.g. `\subsection{1.2b}` and `\subsection{1.2c}` as being part of the answer to question 1.2b.

Compress your answers, i.e. the `.py`, `.tex`, and `.tar`, into a single `.zip` archive, with the name `assignment_1_[studentnumber]_[studentnumber].zip`. You are allowed to work in groups of two. Note that only one student needs to submit to canvas. Make sure that we know exactly who is part of your group.

Introduction

The exercises in this course require you to use Pytorch. This assignment is meant to get you up to speed in case you have not used it before. If you have, this assignment should not take very long. You should be able to complete this assignment using what is given here, the tutorials, and Pytorch's web-pages.

In this assignment, you will train a number of function-approximating networks to familiarize yourself with Pytorch, build practical intuition for machine learning systems, and re-familiarize yourself with some of the fundamentals you've seen in previous courses.

Neural Networks

There are many forms of Neural Networks (NN), which are used for varying applications. In this assignment, you will use the most basic form: a Fully Connected Network (FCN), also called Multi-Layer Perceptron (MLP), where each element is linearly connected to all the elements in the previous layer, followed by a non-linear function (e.g. Tanh, ReLU, etc.). Neural networks are universal function approximators: any function can be approximated with a big enough neural network, given a sufficient number of example pairs of input $\bar{x}$ and desired output $\bar{y}$, both of which can be anything from a single number to multiple megabytes worth of values (e.g. representing an image or a set of radio astronomy traces).

Environment Set-up

As a first step you will need to install Pytorch; go to `https://pytorch.org/get-started/locally/` and follow the instructions.

You should be aware that using both Pytorch and TensorFlow at the same time can throw errors and is not recommended. If you already have TensorFlow installed, make sure to use a separate virtual environment for your torch installation.

Exercise 1.1: MNIST Denoising

For the first task, you will use a network to denoise images from the MNIST dataset. MNIST is a dataset of hand-written numbers; you have been provided with a Pytorch data loader that yields samples from the this dataset as torch tensors. It will provide pairs of clean digits, and the corresponding noisy digit.

Note: None of the questions below require an explanation of more than 2/3 paragraphs, in addition to any graphs that may be required.

(a) Import the data using the dataloader. After importing it, plot a sample of the dataset, to visualise what the data actually looks like. The image should look something like the following:

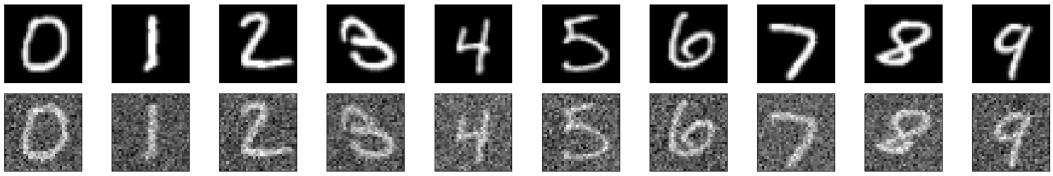

Figure 2: An illustration of the data.

We can now start building the neural network!

(b) Build a fully connected network with linear layers, with a ReLU activation after each hidden layer. Your network should be a class, and its `__init__` function should accept an argument `layer_sizes`, which is an array of numbers describing the number of neurons per layer. This means `YourClassName([32**2, 24**2, 16**2, 32**2])` should initialize a network with 32^2 inputs, two hidden layers with 24^2 and 16^2 neurons, and 32^2 outputs.

(c) Vectorize (i.e. flatten) the 2D image so that it can be used as input for a set of linear layers. Then apply the network, and convert the result back to a 2D image.

(d) For the network to start training, we also need an optimizer. For the implementation, start with using Stochastic Gradient Descent (SGD) and choose an appropriate loss function.

(e) To ensure that your model can actually run, make a prediction with the untrained network using the **test set** (after splitting your data into the training-validation-test sets). Plot the prediction alongside the input data.

Check your understanding:

- What is the difference between SGD and the Steepest Gradient Descent algorithm?
- The clean MNIST values are between -1 and 1. How does that influence your choice of output activation layer?
- What do we mean by an epoch, an iteration, a batch, and a minibatch?
- What is the purpose of splitting the dataset into training-validation-test sets? Why don't we simply use a training-test split?
- Consider the prediction of the untrained model. Why does it look the way it does?

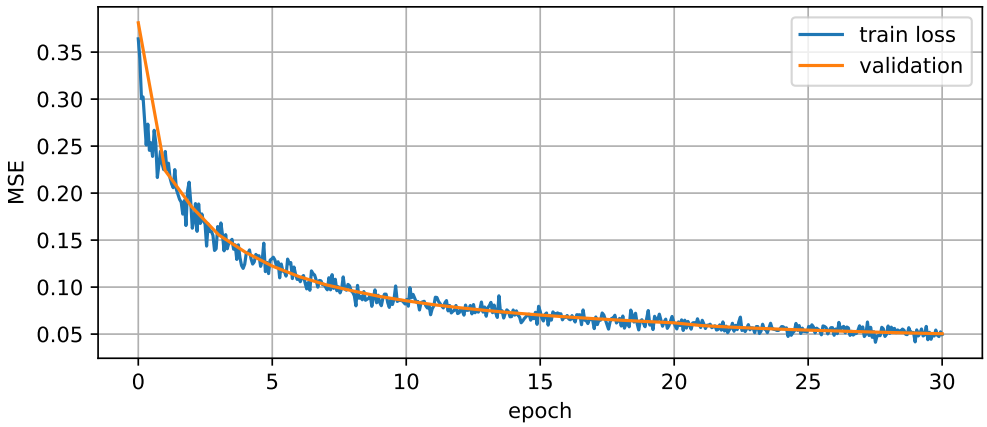

Figure 3: An example of what the losses may look like after 30 epochs.

Time to train your model!

(f) Complete the optimization loop and keep track of the training and validation loss, so you can plot them later. During training, save the weights of your model every N iterations, so you can retrospectively evaluate partially trained versions of your model without having to retrain it.

(g) Plot the training loss and the validation loss in one graph, with appropriate labels (and a legend). Also save the data used to generate your graphs to a unique file, such that you can load and plot them later for comparison without having to re-train your models.

Check your understanding:

- What do these two lines tell you about the quality of your model?

(h) Make a prediction on the test set using your trained model. Make a plot to compare this result to the input and the prediction made on the untrained model.

(i) Now re-run the above experiment at least two more times, both with a bigger network and with a smaller network. Plot the validation loss graphs you obtained with each experiment in one plot, and make a plot comparing the final results on (the same) test set images.

Exercise 1.2: Noise Detection

Humans are quite good at judging whether they are looking at a clean image or a noisy image. This can be seen as a function:

$$f_{\text{clean}}(\text{ }) \quad \rightarrow \quad \{\text{True}, \text{False}\}$$

However, backpropagating through discrete labels is problematic, so we typically formulate such problems as likelihood estimation:

$$p_{\text{clean}}(\text{ }) \quad \rightarrow \quad [0, 1]$$

where we optimize the outputs to be close to 1 for a True label, and close to 0 for a False label. To map this to a True/False answer, we simply threshold at 0.5. In this exercise, you will train a model to detect whether or not the input image was corrupted with noise.

> Check your understanding:
>
> - What do we need to change in the original network architecture to support the function outlined above?
> - How could we enforce the [0, 1] output range?

(a) Create a new class for your noise detection network, and write a new training loop to train it.

(b) As an additional validation metric, track the percentage of validation images that your network correctly classified before and after each epoch.

(c) Train this network. Again plot the train and validation losses, as well as the added validation accuracy metric.

(d) Train at least two variations of your model with a different number of layers and/or neurons per layer, and plot the validation metrics for each.

> Check your understanding:
>
> - What has the largest impact?
> - Does that match your expectations? If not, what could be causing this effect?

Exercise 1.3: Fooling the Detector [GRADED]

This is the graded exercise. You can work together with one other student. If you do, make one shared submission in Canvas, including both of your names and student numbers.

Now you will check if your denoising network can fool your noise detector, and then improve both networks using the numerical equivalent of peer feedback. Before starting this exercise, make sure your detector is performing better than random (i.e. accuracy >50%) and the denoised images from your best denoiser (in exercise (1.i)) are recognizable as digits. This is the graded exercise; when creating plots, label your axes and add legends where appropriate.

(a) Write a function that takes two strings as inputs, representing paths to stored weights. The function should return two objects (i.e. your denoiser and your detector), with the weights loaded from the files.

(b) Load the best version of your denoiser and the best version of your detector from the previous exercises. Apply the denoiser on the test set to create a list of denoised images. Then, apply your detector on the clean images, the noisy images, and the denoised images. Create a figure with three boxplots showing the likelihood of each image being noisy in each set, as estimated by your noise detector.

(c) For each image, calculate whether your detector was "fooled" by your denoiser (i.e. whether the estimated likelihood changed). Make a boxplot of the changes.

(d) Use `plt.subplots` to create a 3x3 grid showing the samples where your detector was "the most fooled", "the least fooled", and a sample that corresponds to the median change in estimated likelihood. Show the clean, noisy and denoised images, and label each image with the estimated likelihoods.

(e) Now use the denoiser to make a better detector! Write a new training loop for your detector that uses your denoiser as an augmentation for the training samples. 50% of the time, use the original noisy training images and 50% of the time, run the noisy inputs through your denoiser first. Remember that your optimizer should only optimize the weights in the detector, not the denoiser.

(f) After training, how do the final validation metrics compare to the ones you found in (2d)? Does that match your expectations? Explain why or why not in ≤ 3 sentences.

(g) Generate figures like in (3b-d) with your new detector. For the figure showing the change in estimated likelihoods, also include a boxplot with the original data from (3c) for easier comparison.

(h) Now use your new detector to make a better denoiser! Write a new training loop for your denoiser that runs the output through your detector. Compute a loss for the detector output, multiply it by $-\alpha$, and then add it to the loss of your denoiser. We want to train our denoiser to fool the detector, so the detection loss should go *up*, not *down*. Remember that your optimizer should only optimize the weights in the denoiser, not the detector. Plot both components of the loss as separate lines.

(i) Depending on the loss functions you chose for both of your networks, different values for α may be appropriate. Explain how you chose your α (optionally support your answer with a figure of experimental results).

(j) Generate figures like in (3g) with your new detector and your new denoiser. For the figure showing the change in estimated likelihoods, also include boxplots with the data from (3c) and (3g) for easier comparison.

Submission format (repeat of info from above)

Code needs to be collated into three standalone files: `exercise_1.py`, `exercise_2.py` and `exercise_3.py`. Which should create all the figures and tabels. We suggest you include a command line argument `--load` in your scripts, which loads your saved results (`.tar`-files) to generate the figures without retraining everything. Make sure the script produces all to-be-loaded files when called without this argument.

Answers that are not code should be provided as a `.tex`-document. Make sure that the answer to each question is clearly labeled using a `\subsection{1.1a}` command. We will parse everything that is placed between e.g. `\subsection{1.2b}` and `\subsection{1.2c}` as being part of the answer to question 1.2b.

Compress your answers, i.e. the `.py`, `.tex`, and `.tar`, into a single `.zip` archive, with the name `assignment_1_[studentnumber]_[studentnumber].zip`. You are allowed to work in groups of two. Note that only one student needs to submit to canvas. Make sure that we know exactly who is part of your group.